Project Log

Continuing the FPGA Matrix-Multiplication Accelerator

TeraNova – FPGA Acceleration

Author	Kai Janssen
Student number	4994817
Period covered	30 September – 30 October 2026
Platform	AMD ZCU102 (Zynq UltraScale+ `xczu9eg-ffvb1156-2-e`)
Tools	Vivado / Vitis / Vitis HLS 2025.1.1

October 30, 2026

Contents

1 Purpose and scope **2**

2 Status summary **3**

3 Chronological log **4**

3.1 30 September 2026 – Setup of the received project 4
3.2 30 September 2026 – Analysis of the previous design 4
3.3 2 October 2026 – Restoring the two IP cores 5
3.4 2 October 2026 – Building the IP and updating the block design 6
3.5 4 October 2026 – Hardware build attempt 6
3.6 7 October 2026 – Pre-build work while waiting for the licence 6
3.7 8 – 28 October 2026 – Organisation of the group work 8
3.8 29 October 2026 – Version control on GitHub 9
3.9 29 October 2026 – Preparing the baseline measurement 9
3.10 29 October 2026 – Simulation of the complete PL data path 10
3.11 29 October 2026 – First improvement: a faster partitioner 11
3.12 29 October 2026 – Baseline 1 frozen, first rung built in 11
3.13 30 October 2026 – The improvement ladder 12

4 Files created and modified **13**

5 Key decisions **14**

6 Open issues and next steps **16**

1 Purpose and scope

This log records the work I carried out to take over the FPGA matrix-multiplication accelerator developed by the previous project group. The overall project goal is to determine whether moving the inverse-scattering calculations of TeraNova's metrology system to the FPGA, and accelerating them in hardware, improves the run time of the system. The work in this period focused on:

1. reproducing the previous group's Vivado and Vitis projects on my own PC;
2. analysing their design and identifying its bottlenecks;
3. restoring two IP cores that were missing from the hand-over, and making the PS–PL interaction robust;
4. preparing a new hardware build;
5. putting the project under version control (GitHub) so that both group members work on the same version;
6. preparing the baseline measurement on the ZCU102 (measurement application and build package);
7. simulating the complete PL data path, including the real HLS RTL, while the board was not available;
8. designing and simulating the first improvement: a faster `matrix_partitioner_B`;
9. organising all improvements as an *improvement ladder* of four rungs, each simulated and prepared for the board on its own git branch.

All work was done in the workspace `C:\PLD`; the files received from the previous group are kept unchanged in a separate folder.

2 Status summary

Item	Status
Vivado project opens in 2025.1.1	**Done**
Vitis platform and application build without errors or warnings	**Done**
Interrupt handling in the application completed	**Done** (not yet tested on board)
Analysis of the previous design and report	**Done**
`RAM_INTERFACE` rewritten (HLS), synthesised for `xczu9eg` and packaged	**Done**
`matrix_partitioner_B` recovered and re-packaged	**Done**
Block design updated to the latest IP builds, validated without errors	**Done**
RTL (GHDL), HLS C and C/RTL co-simulation of the new data path	**Done**
Updated application compiled (against the old hardware export)	**Done**
Unused IP repository `Cezar_repo` removed; all 148 IPs up to date	**Done**
Project under version control (GitHub, `GiganticGnu/TeraNova-FPGA`)	**Done**
Baseline measurement application written and tested in software	**Done** (not yet built in Vitis)
Build package and instructions for the licensed PC	**Done**
Simulation of the full PL path with the real `RAM_INTERFACE` RTL	**Done** (all runs PASS)
Simulation report	**Done**
Faster partitioner (v5a, v5b) designed and simulated	**Done**
Baseline frozen as tag `baseline-1`	**Done**
Ladder rung 1: partitioner v5b in the design (`partitioner-v5b`)	**Done** (simulated, validated)
Ladder rung 2: writer fix, HLS build and co-simulation (`writer-fix`)	**Done** (simulated, validated)
Ladder rung 3: partitioner v6 in the design (`partitioner-v6`)	**Done** (simulated, validated)
Ladder rung 4: burst data path, block design and application (`burst-datapath`)	**Done** (simulated, validated)
Simulation report updated with the ladder; board guide per rung	**Done**
Bitstreams and hardware exports (`.xsa`) of the five rungs	**Open** (licence on Badr's laptop)
Board measurement of the five rungs on the ZCU102	**Open** (board available, builds pending)

Table 1: Status at the end of the logged period.

3 Chronological log

3.1 30 September 2026 – Setup of the received project

Received files. The previous group handed over (via Teams): the Vivado project `S7_integration.zip` (1.27 GB), the Vitis workspace `S7_integration_vit.zip`, five packaged IP cores, the HLS source of the DDR interface and a short README. The README stated that the project depends on the path `C:/PLD/s7_integration`, that the interrupt never worked, and that Vivado/Vitis 2025.1.1 was used.

Workspace. I created `C:\PLD` as workspace so that the absolute paths inside the projects remain valid, and extracted the Vivado project with 7-Zip to `C:\PLD\S7_integration`.

Identification of the contents. Inspection of the archives showed that the target is a Zynq UltraScale+ ZCU102 board (`xczu9eg`), not a Spartan-7 as the name "S7" suggested. The five IP archives turned out to be: `RAM_INTERFACE` (HLS), `MatMul5x5_Acc`, `MatMul5x5_PAR_IF`, `matrix_partitioner_B` and an older, unused `matrix_partitioner`.

IP repositories. From the project file I determined that Vivado expects IP repositories at `../ip_repo` and `../Cezar_repo`. I placed each IP in its own subfolder of `C:\PLD\ip_repo` (two of the archives used the same folder name and would otherwise have overwritten each other) and created an empty placeholder `Cezar_repo` (removed again on 7 October, see Stage 8).

Problem 1 – tool version. On opening, every IP in the block design was locked with status "Newer software version". The cause was that I had Vivado 2025.1 installed, while the project was created in 2025.1.1. I installed 2025.1.1 (with Zynq UltraScale+ MPSoC device support and cable drivers) in `C:\Xilinx\2025.1.1`. Afterwards 143 of the 145 IPs were up to date.

Problem 2 – outdated IP in the hand-over. `RAM_INTERFACE` and `matrix_partitioner_B` remained locked with "IP revision change". The recommended version was *older* than the one in the design (e.g. partitioner Rev. 3 in the design versus Rev. 1 in the archive). By comparing with the copies Vivado keeps in `S7_integration.gen/.../ipshared`, I confirmed that the handed-over archives were outdated. "Upgrade IP" would therefore have downgraded the design, so I did not use it.

Vitis workspace. I extracted `S7_integration_vit.zip` to `C:\PLD\S7_integration_vit` and opened it in the Vitis Unified IDE. The build files showed that the previous group used exactly the same paths, so no path changes were needed. Platform `interrupt_rerun_plat` and application `interrupt_rerun_test` both built successfully. I selected "Always build platform with application" (workspace preference).

Interrupt fix. Reading `helloworld.c` showed why the interrupt never worked: the GIC was configured and the handler connected, but the processor's exception system was never set up. I added `XScuGic_SetPriorityTriggerType()`, `Xil_ExceptionInit()`, `Xil_ExceptionRegisterHandler()` and `Xil_ExceptionEnable()`, plus a check of the connect status. I also removed the two remaining compiler warnings. The original file was kept in `S7_integration_vit\backup`. The application built without warnings.

Lesson learned: one edit was lost because Vitis still had the file open and saved its older copy over the change. Files must be closed in the IDE before they are modified outside it.

3.2 30 September 2026 – Analysis of the previous design

I analysed the HDL, HLS and C sources, the block design, and the timing and utilisation reports of the previous build. The design multiplies two 50×50 matrices of 64-bit integers in 5×5 blocks:

1. the CPU writes A and B into block RAM as 10 000 single 32-bit transactions;
2. `matrix_partitioner_B` reads the blocks element by element (4 clock cycles per element);
3. `MatMul5x5` multiplies two blocks in one clock cycle (1 257 DSPs);
4. an accumulator sums 10 partial products per output block;
5. `RAM_INTERFACE` writes the results to the PL-side DDR4, from where the CPU reads them back one by one.

Main finding. The design is dominated by data movement. Fetching one block pair takes about 100 cycles while multiplying it takes about 5, so the compute unit is idle about 95 % of the time. I estimated the end-to-end run at roughly 2.5–4 ms, and estimated that the proposed improvements (burst transfers instead of CPU single writes, removing the partitioner's wait states, banked memory, data reuse, a narrower pipelined multiplier) could reduce this to roughly 30–50 µs. I also noted that the current accelerator is probably slower than a CPU-only implementation, and that the gain grows with matrix size.

Deliverable. A full report (`previous_group_analysis.tex`, saved in `C:\PLD\report`) with 14 numbered findings, cycle and resource budgets, proposed improvements and expected speed-ups. All figures in it are estimates pending measurements on the board.

3.3 2 October 2026 – Restoring the two IP cores

Decision. Two options were considered: (A) re-package the generated HDL found in the project, or (B) rebuild the IPs from source. I chose **option B**, because:

- it gives the group editable source code instead of opaque generated HDL;
- the improvement plan requires changing `RAM_INTERFACE` anyway;
- the old `RAM_INTERFACE` handshake (plain `ap_none` wires on a free-running block) was not a proper protocol and could capture a result block twice or miss one.

`RAM_INTERFACE` (rewritten in HLS).

- Input is now an `hls::stream` with an `ap_fifo` interface; this maps directly onto the accumulator's existing valid/ready signals (`C_valid` → `data_empty_n`, `C_ready` ← `data_read`).
- Elements are extracted with `.range()` instead of a C union; the inner loop is pipelined (II=1).
- `done` rises only after all writes of the last block are complete.

`matrix_partitioner_B`. The newest VHDL was recovered from the project's `ipshared` folder. Its ports are identical to the archived version, so it only needed re-packaging with core revision 4.

Application update (`helloworld.c`). Separate timing of BRAM load, PL compute, result read-back and CPU-only reference; a 1-second timeout with a GPIO fallback so the program can never hang silently; the start signal is now a toggle so the design can be run repeatedly; two complete runs per execution; build optimisation changed from `-O0` to `-O2`.

Verification (without hardware).

Test	Result
HLS C simulation of `RAM_INTERFACE`: two complete matrices, full 64-bit values	PASS
RTL simulation (GHDL) of the full PL path, test data from `helloworld.c`	PASS
RTL simulation, random full-range 64-bit data	PASS
RTL simulation with a slow writer (back-pressure, 1 500 cycles/block)	PASS, no lost or duplicate blocks
Measured PL run time in simulation	101 008 cycles (≈1.01 ms at 100 MHz)

Table 2: Verification results.

Files delivered to `C:\PLD`: `hls\RAM_INTERFACE` (source, testbench, `build_ip.bat`), `scripts` (three Vivado Tcl scripts), `sim` (GHDL testbench), the updated application and `README_UPGRADE.md`. All replaced files were backed up first.

3.4 2 October 2026 – Building the IP and updating the block design

1. `build_ip.bat` completed: the new `RAM_INTERFACE` (core revision 2114811662) was installed in `C:\PLD\ip_repo\RAM_INTERFACE` with the expected ports.
2. `1_repackage_partitioner.tcl` and `2_update_block_design.tcl` were run in the Vivado Tcl console.
3. Result: **all 145 IPs up to date**, nothing locked. I verified the saved block design: the stream handshake, `done` (interrupt, GPIO channel 2 and ILA), clock, reset, base address and the AXI connection to the DDR4 are all connected as intended, and the address map is unchanged (PL DDR4 at `0x4_0000_0000`).

3.5 4 October 2026 – Hardware build attempt

I ran `3_build_and_export.tcl`. Generating the block design output products succeeded, but all synthesis runs stopped with:

```
ERROR: [Common 17-345] A valid license was not found for feature
'Synthesis' and/or device 'xczu9eg'.
```

The free Vivado edition does not cover the ZU9EG; synthesis, implementation and bitstream generation for this device require a licence. Steps that do not need a licence (HLS, block-design editing, simulation, Vitis software builds, programming the board) all worked. The separate message "Failed to create directory 'C' " appeared at start-up but did not stop the run. The width-mismatch critical warnings in the log were already present in the previous group's design and are harmless.

The group's licence is **node-locked** and registered to one group member. The options identified are: remote access to the licensed PC, having the licence holder run the build script on a copy of `C:\PLD`, or re-hosting the licence to a shared build PC. A new build is only needed when the hardware changes; software changes never need one.

3.6 7 October 2026 – Pre-build work while waiting for the licence

While the licence for the ZCU102 is being set up, I listed which steps of the plan do not need it and carried them out in order. Only synthesis/implementation/bitstream for the `xczu9eg`, the

Vitis platform update (needs the new `.xsa`) and the board run remain blocked.

Stage 1 – Backup. Before changing the projects, I copied `S7_integration` and `S7_integration_vit` to `C:\PLD\backup\S7_integration_2026-10-07` and `...\S7_integration_vit_2026-10-07`.

Stage 2 – HLS build of `RAM_INTERFACE`. `build_ip.bat` ran to completion (log in `hls\RAM_INTERFACE\build_log.txt`). Contrary to expectation, Vitis HLS C synthesis for the `xczu9eg` did *not* need the licence.

Item	Result
C simulation (2 complete matrices)	PASS, 0 errors
Timing, 100 MHz target (10 ns)	7.30 ns estimated; Fmax ≈ 137 MHz
Main loop `ROWS_COLS`	pipelined, II = 1
Latency per 5×5 block (estimate)	41 cycles (0.41 µs)
Resources	2 708 FF, 1 943 LUT, 2 DSP, 0 BRAM (<1 %)
Packaging and installation	IP installed in `ip_repo\RAM_INTERFACE`

Table 3: HLS results of the rebuilt `RAM_INTERFACE`.

The remaining warnings are harmless: one about variable-indexed range selection (`.range()`), power-on initialisation of the three static registers, and that `done` (`ap_none`) cannot be checked automatically by co-simulation.

Stage 3 – C/RTL co-simulation. The first attempt failed before simulating:

```
ERROR: [COSIM 212-345] Cosim only supports the following 'ap_ctrl_none'
designs: (1) combinational designs; (2) pipelined design with II of 1;
(3) designs with array streaming or hls_stream or AXI4 stream ports.
```

This is a tool limitation, not a design error: `RAM_INTERFACE` is free-running (`ap_ctrl_none`), takes several cycles per block and writes through `m_axi`. To still verify the generated RTL, I added a co-simulation-only build: `RAM_INTERFACE.cpp` selects `ap_ctrl_hs` when the macro `COSIM_HS` is defined (otherwise `ap_ctrl_none`, so the packaged IP is unchanged), and the new `hls_config_cosim.cfg` sets `-DCOSIM_HS` and uses its own work folder (`work_cosim`). `run_cosim.bat` runs synthesis and co-simulation of this variant.

Result: **C/RTL co-simulation PASS** – all 2×100 blocks written correctly by the generated RTL in xsim, and the `done` check passed. The simulated time per block was about 780 ns (≈78 cycles instead of the 41 estimated, because of waiting for AXI write responses), i.e. about 7.8 µs to write a complete 50×50 result.

Stage 4 – Re-packaging the partitioner. `1_repackage_partitioner.tcl` ran without errors (integrity check passed). It reported "Old core revision: 4": the re-packaging had already been applied on 2 October, so the run only re-saved the same revision.

Stage 5 – Block design. Before running `2_update_block_design.tcl` I inspected the saved block design. It already contained everything that script does (partitioner revision 4; the new `RAM_INTERFACE` ports wired to the accumulator; `done` to GPIO, interrupt and ILA; `m_axi_gmem` to `axi_smc/S01_AXI`; address segment at `0x4_0000_0000`, 512 MB). Re-running it would delete and recreate a correctly wired cell, so I did not run it. The only issue was that `RAM_INTERFACE_0` showed as *locked*, because today's HLS build has a new core revision. As the ports are identical, I wrote `2b_upgrade_after_rebuild.tcl`, which only upgrades the two IPs, validates and saves. Result:

- `RAM_INTERFACE_0` upgraded from revision 2114811662 to 2114818764;

- `validate_bd_design`: no errors (only the known SmartConnect, AWUSER/ARUSER width and debug-net warnings);
- checked in the saved `accu_to_DDR.bd`: all `RAM_INTERFACE_0` connections and the address segment unchanged.

Stage 6 – Application build. The updated `helloworld.c` (from 2 October) had not been built yet. I built `interrupt_rerun_test` in Vitis against the existing (old) platform: **build successful**, no errors (text 93 748 B, data 9 576 B, bss 582 128 B). The code only uses hardware names that remain the same in the new design, so it is expected to build unchanged against the new `.xsa`.

Stage 7 – Board bring-up preparation. None of these steps needs the licence, but most need the board. Possible now: installing the JTAG cable drivers and the Silicon Labs CP210x USB-UART driver, and preparing a serial terminal profile (115 200 baud, 8N1, no flow control). With the board: check that four CP210x COM ports appear (the lowest is PS UART0), set SW6 to JTAG boot (all ON), and check that the Hardware Manager detects the `xczu9eg`.

Stage 8 – Removing the unused IP repository. The project still listed `../Cezar_repo` as a second IP repository. No IP in the block design came from it; the folder only held my placeholder README. In the Vivado Tcl console:

```
set_property ip_repo_paths C:/PLD/ip_repo [current_project]
update_ip_catalog
```

Vivado then loaded only `c:/PLD/ip_repo`. I checked the saved `S7_integration.xpr` (only one `IPRepoPath` left), searched the project file, block design, IP configuration files and scripts for remaining references to the folder (none; the hierarchy block `Cezars_pin_paradise` only shares the name and is unrelated), and then deleted `C:\PLD\Cezar_repo`. `report_ip_status` afterwards: **148 IPs, all up to date**, none locked. Three Constant IPs inside the SmartConnects are marked "Up-to-date (discontinued)", which needs no action. The four custom IPs (`MatMul5x5_Acc` Rev. 4, `MatMul5x5_PAR_IF` Rev. 2, `matrix_partitioner_B` Rev. 4, `RAM_INTERFACE` Rev. 2114818764) are all found in `ip_repo`.

Stage 9 – Generate Output Products (licence check). On opening the project, Vivado warned that the generated files of `accu_to_DDR_axi_smc_0` were missing. I ran *Generate Output Products* (out of context per IP, 12 jobs). Three out-of-context runs failed with `[Common 17-345] A valid license was not found for feature 'Synthesis' and/or device 'xczu9eg'`: `accu_to_DDR_axi_smc_0`, `accu_to_DDR_matrix_partitioner_B_0_0` and `accu_to_DDR_RAM_INTERFACE_0_1`. These are exactly the IPs without a cached synthesis result (the two rebuilt IPs and the SmartConnect affected by the upgrade); all others were reused from the cache. This confirms the 4 October finding: on my PC, even generating output products needs the licence. It is not related to Stage 8. The runs will be redone by `3_build_and_export.tcl` on the licensed PC.

Generation also gave twelve `[BD 41-2383] Width mismatch` critical warnings: six connections, each reported by validation and again by generation. They concern the address path of the previous group's design: partitioner `bram_addr_A/B` (12 bits) → `mult_gen_*/A` (18 bits), and `mult_gen_*/P` (21 bits) → `Part0..3/addrb` (32 bits). In every case the narrower side covers the full address range (12 bits = 4096 values for 2500 elements per matrix), and the unconnected upper bits are tied to 0 in synthesis. They are harmless and were not caused by the upgrade.

3.7 8 – 28 October 2026 – Organisation of the group work

Licence. Per the supervisor, the node-locked Vivado licence for the `xczu9eg` can only be held by one group member. It is installed on Badr's laptop. From then on, the work is split: I do

everything that does not need the licence (HLS, simulation, software, analysis, board runs), and the licence-bound steps (synthesis, implementation, bitstream) run on Badr's laptop.

One shared version. Badr and I each had our own build in `C:\PLD`. We agreed to continue from my restored version as the shared baseline and build on top of it.

Board. In late October the group got access to the ZCU102 board.

3.8 29 October 2026 – Version control on GitHub

To work on one version without passing zip files around, I put `C:\PLD` under git and connected it to the GitHub repository `GiganticGnu/TeraNova-FPGA` (branch `main`). The initial commit, *"Baseline: restored working build"* (`2e936e3`, 29 October, 12:01), contains the restored build as described in this log.

The `.gitignore` in `C:\PLD` follows one rule: commit sources, ignore everything the tools regenerate.

- **Tracked:** `S7_integration.xpr` and `S7_integration.srcs` (block design, IP configurations, constraints), the current `interrupt_rerun.xsa`, the packaged IPs in `ip_repo`, the HLS sources and scripts, the Vitis component files and application sources, the simulation sources, the scripts and the reports.
- **Ignored:** Vivado's generated folders (`.gen`, `.runs`, `.cache`, `.hw`, `.sim`, `.ip_user_files`), logs and journals, old `.xsa` exports, Vitis build output (`build`, `export`, `_ide\bitstream`, `.elf`), HLS work folders, GHDL output, and the local `backup` folders, which git history now replaces.

3.9 29 October 2026 – Preparing the baseline measurement

The goal is to measure how fast the system currently is on the ZCU102, before any improvement. All improvements are then judged against these numbers.

Two baselines. The Vitis platform still points at the previous group's original export (`interrupt_rerun.xsa` of 30 September). Programming and running the board needs no licence. So I planned two baselines:

- **Baseline 0:** the previous group's bitstream, unchanged. It can be measured as soon as the board is available.
- **Baseline 1:** the restored design (rebuilt `RAM_INTERFACE`, partitioner revision 4), once Badr has built its bitstream. This is the reference for the improvements. Its PL compute time should agree with Baseline 0, because the data path did not change.

Measurement application. I wrote a measurement version of `helloworld.c` (`C:\PLD\baseline\helloworld.c`). It does the same work as the normal application and uses only hardware names that exist in both the old and the restored design. Compared with the normal application it:

- runs the PL 10 times instead of twice and prints minimum, average and maximum per phase;
- measures with 10 ns resolution (the A53 generic timer) instead of whole microseconds;
- measures the cost of one PS→BRAM write, one BRAM read and one PL-DDR4 read ("latency probes"), so the load and read-back totals can be explained;
- repeats the CPU-only reference and reports the cold and the best (warm-cache) time;
- detects `done` both by interrupt and by polling GPIO channel 2, so a compute time is measured even if the interrupt path does not work;

- prints one `CSV,` line per run for a spreadsheet, and the compiler optimisation level of the build.

Verification: it compiles without warnings at `-O0` and `-O2` against stub versions of the Xilinx headers, and its logic was tested against a simulated board in four cases: interrupt works, only GPIO works, nothing arrives, and one wrong result. Each case gave the expected output. It still has to be built in Vitis and run on the board. A `results` folder (with a README for the log files) is ready next to it.

Build package for the licensed PC. Badr only needs the Vivado side: `S7_integration`, `ip_repo` and `scripts`, at exactly `C:\PLD`. His steps: rename his own old `C:\PLD`, unpack the package, check Report IP Status, run `3_build_and_export.tcl`, and save utilisation and timing reports (`report_utilization`, `report_timing_summary`). He then sends back `interrupt_rerun.xsa` and the two reports. On my side the Vivado project stays frozen until the `.xsa` comes back. Everything is written down in a separate step-by-step baseline guide.

Correction to the setup guide. After a platform update the bitstream is found in `interrupt_rerun_plat\export\interrupt_rerun_plat\hw\sdt\interrupt_rerun.bit`. The setup guide gives this path without the `sdt` folder.

3.10 29 October 2026 – Simulation of the complete PL data path

Working from home without the board, I simulated everything inside the PL. The results are written up in a separate report (`report\simulation_report`, also as an Overleaf project).

Testbenches. The functional testbench `tb_chain` and the HLS testbench already existed. I added (all in `C:\PLD\sim`):

- `tb_chain_perf`: `tb_chain` plus passive monitors on every valid/ready handshake that count cycles per stage;
- `tb_chain_rtl`: the data path with the *real* `RAM_INTERFACE` RTL generated by Vitis HLS, writing through `m_axi` into a new AXI4 memory model (`axi_mem_model.vhd`) with adjustable write-response latency L. With `RUNS=2` it does a second complete run without reset;
- `run_all.py`, which compiles and runs all cases and writes `results.json`.

One line of the generated `RAM_INTERFACE_gmem_m_axi.vhd` uses a VHDL-2019 construct that GHDL does not accept. In the simulation copy (`sim\hls_rtl`) it is replaced by an equivalent VHDL-2008 function, marked `SIM PATCH`. The packaged IP is unchanged.

Results (GHDL 4.1.0; all runs give the correct 50×50 result):

Result	Value
PL run, writer model	101 008 cycles (1.010 ms)
PL run, real `RAM_INTERFACE`, $L = 20$	101 186 cycles (1.012 ms)
Partitioner: interval between block pairs	101 cycles (25 elements × 4 states + 1)
Multiplier busy	5 000 cycles (4.95 %)
Writer: AXI bursts	504 bursts of 1–5 beats (one per block row; 4 split at 4 KB)
Restart without reset	PASS, second run identical
Random 64-bit data, slow writer, throttled write channel	PASS

Table 4: Main simulation results of the current design.

These confirm findings F3, F6, F7 and F11 of the analysis report with exact numbers. The simulation also found something new. The generated `RAM_INTERFACE` waits for the write response after every row of 5 values, so it needs $85 + 5L$ cycles per output block. This explains why co-simulation measured 78 cycles per block where HLS estimated 41. With the current partitioner the writer stays hidden up to $L = 185$ cycles.

3.11 29 October 2026 – First improvement: a faster partitioner

I wrote two drop-in versions of `matrix_partitioner_B` (`sim\variants`). Both have the same entity, ports, block order and output packing as revision 4:

- **v5a:** revision 4 with `WAIT1` and `WAIT2` removed (2 cycles per element);
- **v5b:** pipelined fetch, a new BRAM address every cycle, each element captured two edges later (1 cycle per element).

Both need the BRAM read latency to be 1 cycle and the address multipliers to add no delay. I checked this in the `.xci` files of the block design. All four BRAMs have `C_READ_LATENCY_B = 1` without output registers, and both `mult_gen` instances have `C_LATENCY = 0`.

	rev. 4	**v5a**	**v5b**
Cycles per block pair	101	51	27
PL run, real `RAM_INTERFACE`, $L = 20$ (cycles)	101 186	51 186	27 186
Speed-up of the PL part	1.00×	1.98×	3.72×
Largest latency L the writer can hide (cycles)	185	85	37
All functional tests incl. restart	PASS	PASS	PASS

Table 5: Simulated partitioner versions (`run_variants.py`, 63 runs).

v5b brings the PL run from 1.012 ms to 0.272 ms. It reaches that speed only if the result memory confirms a write within 37 cycles; above that, the result writer sets the pace for every version. The variants are not yet in `ip_repo`: the hardware design is unchanged until the group decides to build one.

3.12 29 October 2026 – Baseline 1 frozen, first rung built in

Commit and tag. I committed the measurement application, the new testbenches, the partitioner variants (simulation only), the simulation report and this log (`3c52095`, 15:20) and tagged that commit `baseline-1`. This is the version Badr builds for Baseline 1. My first attempt put the tag on the previous commit, because the commit step had been skipped. I deleted the tag locally and on GitHub, committed, and tagged again; the click-by-click guide now checks the commit before tagging.

Partitioner v5b. On a new branch `partitioner-v5b` I put v5b into `ip_repo` as core revision 5 and upgraded it in the block design with `scripts\4_partitioner_v5b.tcl`. The ports are unchanged, so no rewiring was needed. Validation gave no errors. Commit `5c49045`.

Wrong Vivado version. Once I opened the project in Vivado 2025.1 instead of 2025.1.1 by mistake, and every IP was locked again. I closed it without saving and restored `ip_repo` with `git restore`. Since then every script stops at once unless it runs in 2025.1.1 (it checks `XILINX_VIVADO`), and it also checks that the right git branch is checked out.

3.13 30 October 2026 – The improvement ladder

Method. To see the effect of each change on its own, both in simulation and later on the board, the improvements form a *ladder*. Each rung adds one change on top of all earlier ones, in a fixed order, on its own git branch. The branches are stacked (`baseline-1` → `partitioner-v5b` → `writer-fix` → `partitioner-v6` → `burst-datapath`); none is merged into `main`, which stays at the baseline until all rungs have been measured. After every change `sim\run_ladder.py` simulates the whole PL data path again with the real RTL of every block, at the nominal write-response latency $L = 20$ and at $L = 100$, with two data sets and a restart without reset. The full results are in the updated simulation report.

Rung	Change	$L = 20$	$L = 100$	Commit
0	Baseline (tag `baseline-1`)	101 186	101 586	`3c52095`
1	Partitioner v5b, pipelined fetch	27 186	58 576	`5c49045`
2	Writer with overlapped write responses	27 071	27 151	`70ba245`
3	Partitioner v6, on-chip copy of A and B	10 052	18 052	`b056560`
4	Burst data path through `S_AXI_HP0_FPD`	10 073	18 153	`b99091c`

Table 6: Simulated PL run time per rung (clock cycles at 100 MHz). All rungs pass all functional tests.

Rung 2 – writer fix (branch `writer-fix`). I rewrote `RAM_INTERFACE` with an explicit burst interface (`hls::burst_maxi`): per 5×5 block it issues the five write requests, then the 25 values, and only then collects the five responses, with up to 8 writes in flight. I ran `build_ip.bat` and `run_cosim.bat` (both PASS) and upgraded the IP in the block design with `2b_upgrade_after_rebuild.tcl`. The new generated RTL was copied into the ladder (with the same `SIM PATCH` line as before). The writer now needs about $L + 52$ cycles per block instead of $85 + 5L$, so rung 1 keeps its speed up to $L \approx 215$ cycles instead of 37.

Rung 3 – partitioner v6 (branch `partitioner-v6`). v6 copies A and B once into 25 internal banks per matrix and then reads a whole block of A and B in one cycle, so every block is fetched once instead of ten times. Same ports as revision 4, packaged as core revision 6. For this and later partitioner steps I made one generic install script (`install_partitioner.tcl`, called by `5_partitioner_v6.tcl`). The run drops to 10 052 cycles; the writer (74–75 cycles per block) is the limit again.

Rung 4 – burst data path (branch `burst-datapath`). A, B and C now live in the processor's DDR (DDR high, `0x8_0000_0000`, `0x8_0000_5000`, `0x8_0000_A000`; this region is not used by the linker script). Partitioner v7 reads A and B itself through a 128-bit AXI4 read master in bursts of up to 256 beats, and `RAM_INTERFACE` writes C there; both go through a new SmartConnect into the high-performance port `S_AXI_HP0_FPD`. For simulation I wrote an AXI4 read-slave model that also checks the protocol rules. The PL time stays at 10 073 cycles; the gain is on the CPU side, which no longer writes 10 000 single values into BRAMs.

- **Block design.** `scripts\6_burst_datapath.tcl` removes the four BRAMs, their controllers, `smartconnect_0`, the two `mult_gen` blocks, the two enable constants and the old partitioner. It adds v7 as an RTL module reference (`hdl\partitioner_v7.vhd`, a copy of the simulated file with interface attributes), the constants `A_BASE`/`B_BASE`, the SmartConnect `axi_smc_hp0`, and enables HP0 at 128 bits. The clock and the PL DDR4 controller stay as they are, so this rung only changes the data path. The GPIO address (`0xB000_0000`) is unchanged.
- **First run of the script failed**: Vivado does not accept a VHDL-2008 file as the top of a module reference. The file is plain-VHDL compatible (checked with GHDL), so the

script now marks it as VHDL. The script also creates the new partitioner *before* it removes anything, so a failure leaves the design intact. I closed without saving, restored the project file with `git restore S7_integration` and ran the script again. Validation: no errors and no critical warnings. The remaining warnings (AXI user-signal widths, dangling nets of removed cells, an existing ILA clock association) were checked and are harmless.

- **Application.** HP0 is not cache coherent, so the new application (`app\burst_datapath\helloworld_burst.c`) flushes A and B from the data cache before start and invalidates C after `done`. It reports these steps in the same CSV columns as the baseline ("load" is now the cache flush) and in an extra `CSV,burst` line. It was tested against a model of a non-coherent cache: all runs pass, and without the flush the start-up buffer check fails, as it should. `scripts\6b_app_burst.bat` puts it into the Vitis application and removes `bram_write.c/.h`, which no longer compile without BRAMs.

Committed and pushed as `b99091c`.

Documentation. I updated the simulation report with the ladder and wrote a board guide that explains, rung by rung, how each branch is built, loaded onto the ZCU102 and measured.

4 Files created and modified

Path (under `C:\PLD`)	Content
`ip_repo\*`	IP repository (5 IPs, one folder each)
`Cezar_repo\README.txt`	Placeholder for the missing repository; folder deleted and removed from the project on 7 Oct
`S7_integration\S7_integration.xpr`	IP repository list reduced to `../ip_repo` (7 Oct)
`hls\RAM_INTERFACE\*`	Rewritten HLS source, testbench, config, build script
`hls\RAM_INTERFACE\hls_config_cosim.cfg`, `run_cosim.bat`	Co-simulation build (`ap_ctrl_hs` via `COSIM_HS`), 7 Oct
`hls\RAM_INTERFACE\build_log.txt`, `cosim_log.txt`	Logs of the 7 Oct HLS build and co-simulation
`ip_repo\matrix_partitioner_B\*.vhd`	Newest partitioner VHDL (revision 4)
`scripts\1_ ... 3_*.tcl`	Re-package, block-design update, build and export
`scripts\2b_upgrade_after_rebuild.tcl`	Upgrade-only update after an IP rebuild (7 Oct)
`sim\*`	GHDL testbench of the complete PL data path
`S7_integration_vit\...\src\helloworld.c`	Updated application
`S7_integration_vit\...\src\UserConfig.cmake`	Optimisation `-O0` → `-O2`
`report\previous_group_analysis.tex`	Analysis report of the previous design
`README_UPGRADE.md`	Step-by-step upgrade guide
`.gitignore`	Git rules: sources tracked, generated files ignored (29 Oct)
`baseline\helloworld.c`	Baseline measurement application (29 Oct)
`baseline\results\README.txt`	Naming of the board log files and CSV fields

Path (under C:\PLD)	Content
sim\tb_chain_perf.vhd, run_perf.py	Testbench with handshake monitors (29 Oct)
sim\tb_chain_rtl.vhd, run_rtl.py, axi_mem_model.vhd	Testbench with the real RAM_INTERFACE RTL and an AXI4 memory model
sim\hls_rtl*.vhd	Simulation copy of the generated RAM_INTERFACE RTL (one SIM PATCH line)
sim\run_all.py, results.json	All simulations of the current design and their results
sim\variants*.vhd	Partitioner versions v5a and v5b
sim\run_variants.py, variants_results.json	Simulations of the partitioner versions and their results
sim\make_figures.py, README_perf.md	Report figures; description of the new testbenches
report\simulation_report*	Simulation report (LaTeX, figures, PDF)
scripts\4_partitioner_v5b.tcl	Rung 1: v5b as core revision 5 (29 Oct)
hls\RAM_INTERFACE\ RAM_INTERFACE.cpp/.hpp	Rung 2: writer with hls::burst_maxi (30 Oct)
scripts\install_partitioner.tcl, 5_partitioner_v6.tcl	Rung 3: generic partitioner install; v6 as revision 6
hdl\partitioner_v7.vhd	Rung 4: hardware copy of partitioner v7 (module reference)
scripts\6_burst_datapath.tcl, 6b_app_burst.bat	Rung 4: block-design change; application swap
app\burst_datapath\helloworld_burst.c	Rung 4: measurement application with cache maintenance
sim\variants*_v6.vhd, *_v7.vhd	Partitioners of rungs 3 and 4
sim\ladder.json, run_ladder.py, ladder_results.json, ladder.md	Improvement ladder: definition, runner, results
sim\steps\2_writer_fix*	Generated RTL of the new writer for simulation
sim\axi_rd_mem_model.vhd, tb_chain_axi.vhd	AXI4 read-slave model and testbench of rung 4
sim\make_ladder_figures.py	Ladder figures for the simulation report
report\TeraNova\board_guide*	Board guide: building and measuring every rung (30 Oct)
backup*	Old partitioner VHDL and RAM_INTERFACE IP
backup\S7_integration*_2026-10-07	Full copies of the Vivado project and Vitis workspace (7 Oct)
S7_integration_vit\backup*	Original and pre-upgrade application files

5 Key decisions

Decision	Rationale
Workspace at C:\PLD	Keeps the previous group's absolute paths valid without editing project files.
Install Vivado 2025.1.1	Exact version match prevents IP locking and upgrade issues.

Decision	Rationale
Do not use "Upgrade IP" for the two locked IPs	The offered version was older than the design's; upgrading would have downgraded the hardware.
Rebuild `RAM_INTERFACE` from source (option B)	Editable code, a correct handshake, and a starting point for the planned improvements.
Verify by simulation before building	Hardware builds are long and currently licence-bound; simulation catches functional errors cheaply.
Co-simulate a separate `ap_ctrl_hs` variant	Co-simulation does not support this `ap_ctrl_none` design; a macro-selected variant verifies the same data path and `m_axi` writes without changing the packaged IP.
Do not re-run `2_update_block_design.tcl`; use an upgrade-only script	The block design was already rewired; only the IP revision changed, and an upgrade keeps the existing connections.
Remove `Cezar_repo` from the project	No IP came from it; a second, empty repository only adds a source of confusion and a risk of picking up stale IP versions later.
Licence-free work on my PC, builds on Badr's laptop	The licence can only be held by one member; a build is only needed when the hardware changes.
My restored version is the shared baseline	One version avoids two diverging copies of `C:\PLD`.
GitHub with a sources-only `.gitignore`	Both members work on one history; generated folders are large and are recreated by the tools.
Measure two baselines (old bitstream, restored design)	Baseline 0 needs no build and can be measured at once; agreement with Baseline 1 shows the restoration did not change the timing.
Simulate the real HLS RTL, not only a model	The model hid the per-row write-response wait in the generated `RAM_INTERFACE`.
Prefer partitioner v5b over v5a	Same build effort, almost twice the gain (3.72× vs 1.98×).
Keep the variants out of `ip_repo` on `main`	The baseline stays identical to the version being measured; each variant goes into `ip_repo` only on its own branch.
Improvement ladder: one change per rung, one branch per rung, no merges into `main`	Every change can be simulated and measured on its own, and compared with the rung before it.
Scripts check the Vivado version and the git branch	Opening the project in 2025.1 locks every IP; running a script on the wrong branch would change an earlier rung.
Writer fix before partitioner v6	With v5b the writer limited the speed above $L = 37$; fixing it first makes rungs 1 and 3 independent of the unmeasured memory latency.
Rung 4 keeps the 100 MHz clock and the PL DDR4 controller	Only the data path changes, so the measured difference belongs to that change alone.
Partitioner v7 as an RTL module reference, not as packaged IP	Its ports differ completely from revisions 4–6; a module reference infers the AXI master from the VHDL attributes and needs no re-packaging.

6 Open issues and next steps

Status 30 October. Items 1, 6, 7 and 9 of the list below (written on 29 October) are done: the work is committed, v5b and the writer fix are rungs 1 and 2, and the burst data path is rung 4. What remains:

1. **Builds**: Badr builds the five rungs in ladder order (`baseline-1`, `partitioner-v5b`, `writer-fix`, `partitioner-v6`, `burst-datapath`) and commits each `.xsa` to its branch, as described in the board guide. WNS must be ≥ 0 for each.
2. **Board runs**: run every rung with its own application and save the logs (board guide). Replace the CPU estimates in `ladder.json` by the measured values and compare the PL compute times with the simulation.
3. **Next rungs** (after the measurements): writer with longer bursts, a second read port, a pipelined multiplier.
4. **Setup guide**: correct the bitstream path and refer to the board guide.

List of 29 October:

1. **Commit the new work.** The baseline application, the new testbenches, the variants and the simulation report were made after the initial commit and are not yet in the repository.
2. **Board preparation**: install the JTAG and CP210x USB-UART drivers; check that four CP210x COM ports appear (the lowest is PS UART0), set SW6 to JTAG boot (all ON) and check that the Hardware Manager detects the `xczu9eg`.
3. **Baseline 0**: build the measurement application in Vitis and run it on the ZCU102 with the existing bitstream (no licence needed).
4. **Build**: Badr runs `3_build_and_export.tcl` on the restored design and returns `interrupt_rerun.xsa` plus the utilisation and timing reports (WNS ≥ 0). To decide: build from a GitHub clone or from the 7-Zip package. A clone does not contain the ignored generated folders (`.gen`, `.runs`, `.cache`); the build script regenerates them, which has to be confirmed on the first build. The new `.xsa` can come back through the repository, because `interrupt_rerun.xsa` is tracked.
5. **Baseline 1**: update the Vitis platform with the new `.xsa`, check the bitstream date, measure, and compare the PL compute time with the simulated 1.012 ms.
6. **Improve**: put partitioner v5b in `ip_repo` (core revision 5), have it built and measured (expected 0.272 ms PL compute).
7. **Writer**: change `RAM_INTERFACE` so that it no longer waits for a write response after every row, and verify it in `tb_chain_rtl`; otherwise v5b only reaches its speed if the memory latency is at most 37 cycles.
8. **Setup guide**: correct the bitstream path (`hw\sdt`) and add the GitHub workflow.
9. **Then**: the burst-based data path from the analysis report (PL master on `S_AXI_HP0_FPD`).